

Architecture pattern	Input res.	p	merge	Tokens / image
Fixed-resolution ViT (CLIP-L/14 @ 336)	336 × 336	14	none	24 × 24 = 576
Native-resolution ViT + 2×2 merge	1280 × 720	14	2 × 2 (28 px per token)	≈ 46 × 26 ≈ 1,196
Same, downscaled	640 × 360	14	2 × 2	≈ 23 × 13 ≈ 299
Tiling "any-res" schemes	Multiple tiles + thumbnail	14	varies	1,500–3,000+

That arithmetic produces the slide's "250–1000+" range. Resolution is a **quadratic** cost lever: halving both sides cuts tokens by about 4×.

Why sampling is unavoidable. Unsampled video is:

$$\text{tokens/min} = \text{fps} \times 60 \times N_{\text{tokens}} = 30 \times 60 \times 1,196 \approx 2.15 \text{ M tokens per minute}$$

That exceeds most context windows many times over, and self-attention cost grows as $O(n^2)$. At **0.5 fps** and 640 × 360, one minute costs $30 \times 299 \approx \mathbf{9k \text{ tokens}}$, a 240× reduction. Some architectures also merge *temporally adjacent* frames (temporal patch size 2) into shared tokens, which halves video token counts again.

Sampling strategies.

Strategy	Rule	Good for	Risk
Uniform	Every Δt seconds	Predictable cost; talking-head meetings	Misses short events (temporal aliasing)
Scene-change	Frame when pixel/histogram difference > θ	Slides, cuts, presenter changes	Floods on camera motion; misses slow changes
Keyframe (I-frame)	Decoder keyframes only	Very fast extraction	Encoder-dependent spacing
Adaptive / two-pass	Cheap detector scores frames; VLM on top-scored ones	Rare-event search	More components

Temporal Nyquist intuition. To reliably capture an event of duration d , sample with period $\Delta t \leq d/2$. A 2-second gesture needs ≥ 1 fps; a 20-second slide transition is safe at 0.1 fps.

⚙️ Production Implementation

```
# frame_budget.py: plan sampling to meet a token budget, then sample with scene awareness
import math
import cv2
import numpy as np

def tokens_per_frame(w: int, h: int, patch: int = 14, merge: int = 2) -> int:
    unit = patch * merge
    return math.ceil(w / unit) * math.ceil(h / unit)

def plan(duration_s: float, w: int, h: int, token_budget: int, max_fps: float = 2.0):
    """Largest fps (<= max_fps) whose total visual tokens fit the budget."""
```